Text Priors Transfer Little: Label Sharing and Test-Pool Calibration for Cold-Start Predictive AI Evaluation

Yaroslav Rogoza*
ITMO University

Abstract

The Predictive AI Evaluation Competition (PAIEC) asks for the probability that an AI system answers a task correctly on private benchmarks. Predictions start with no labels and then see a few revealed labels for each system and benchmark pair, called a group. We trained a prior from task and system text on 166 public benchmarks, and on held-out benchmarks it improved the score by only 0.1%. The gain comes from sharing revealed labels across systems on the same benchmark and across similar items, and from matching our simulator of the official draws to the hidden test pool. The feedback of the official constant-prediction baseline shows that test groups lie much farther from 50% accuracy than training groups. Models that raise their predictions for recent systems scored worse before any label than the same model without this shift (Brier score 0.275 and 0.284 against 0.249), which points to most test groups lying below 50%. Both facts changed our final model. It uses a wider prior and no release-date or model-name offsets, and in the calibrated simulator it scores about 3% better than a Beta-binomial baseline with a uniform prior that uses only the revealed labels. Single official runs are too noisy to confirm differences of this size.

1 Introduction

In PAIEC a submission receives a description of an AI system, the *subject*, and of a task, the *item*, and returns the probability of a correct answer [1]. The test benchmarks are private. For each subject and benchmark pair, which we call a *group*, the platform reveals from 0 to 31 labels and scores the predictions on the remaining items with the Brier score. The score is the area under this learning curve (ALC), lower is better, and each run scores a random draw of 8 or 9 groups (Section 2).

A first idea is to learn a prior from item and subject text and update it with the revealed labels. Our earlier entries V2 and V3 did this with a frozen E5 encoder [6] and four or five training benchmarks [2]. Before any label, V2's official Brier score averaged 0.249, no better than a constant prediction of 0.5. We retrained the prior on 166 benchmarks with many times more responses, and on held-out benchmarks it improved ALC by only 0.1% (Section 6.1).

What helps is the labels of other subjects. Our model shares a benchmark offset across all subjects in a draw and moves an item's prediction when other subjects answered that item or a similar one (Section 4). On the training distribution this lowers ALC by 1.8% against a Beta-binomial baseline that uses only the group's own labels, and 95% of the gain remains without the text prior (Figure 1).

We then matched the simulator to the hidden test pool. The official baseline predicts one constant per group, so its feedback reveals how far each group's accuracy lies from 50%. Test groups lie much

*Technical report for the NeurIPS 2026 Predictive AI Evaluation Competition (PAIEC). Code: `https://github.com/123yaroslav/paiec2026`.

Preprint.

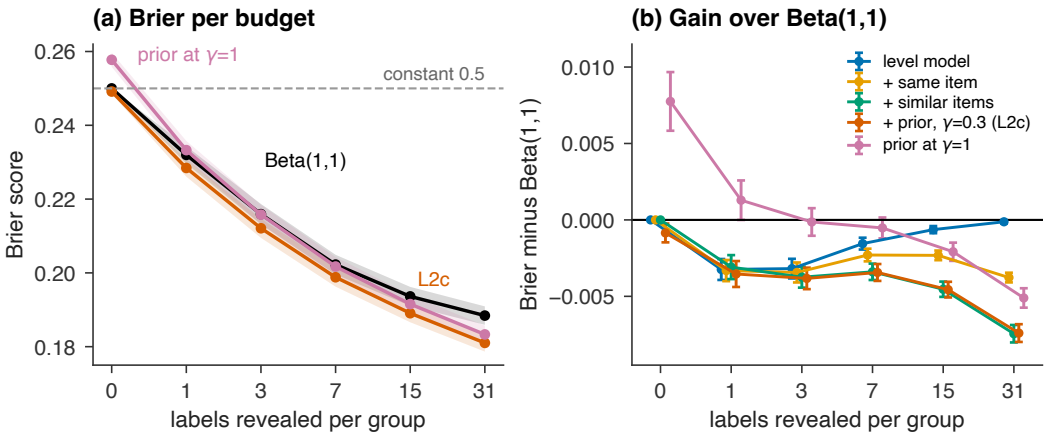


Figure 1: **Label sharing carries the gain. The text prior adds little at the chosen weight and hurts at full weight.** (a) Brier score per budget for the Beta(1,1) baseline, the narrow model L2c with the text prior at weight $\gamma = 0.3$, and L2c at full weight $\gamma = 1$. (b) Difference from Beta(1,1) as each part is added. Simulator on the training distribution, 600 fresh draws, 95% intervals.

farther from 50% than training groups, and reweighting the simulator to match reproduces the official baseline curve (Figure 2a). Under these weights a wide prior, which allows large benchmark and subject offsets, beats the narrow prior we had chosen (Section 6.3).

This feedback cannot tell whether a group lies above or below 50%, but our runs of models with release-date and model-name offsets give a strong hint. The offsets raise predictions for recent systems and helped on training data, where most recent systems score above 50%. On the platform every run of these models scored worse before any label than every run of the same model without offsets, 0.275 and 0.284 against 0.249 on average. This points to most test groups lying below 50%, so our final model, L3, has no offsets (Section 6.4). This report makes three contributions.

1. A measured negative result. A frozen-encoder text prior trained on 166 benchmarks transfers almost nothing to unseen ones (Section 6.1).
2. A label-sharing model that accounts for most of the improvement (Sections 4 and 6.2).
3. A way to calibrate a simulator to a hidden test pool from aggregate feedback, which changed our model choice twice (Sections 5 to 6.4).

2 Task and official draws

A subject has eight text fields such as model name, harness and reasoning effort. An item has its content, metadata and an anonymous benchmark alias. The function `predict([subject, item], labeled)` returns a probability and sees every label revealed so far in the draw, for all groups. With $B_g(k)$ the Brier score of group g after k revealed labels,

$$\mathrm{ALC}_g = 0.1\, B_g(0) + 0.2\,\big[B_g(1) + B_g(3) + B_g(7) + B_g(15)\big] + 0.1\, B_g(31), \tag{1}$$

and the score of a draw is the mean of ALC_g over its groups.

All our formative feedback tables share one structure (Appendix G). A draw holds 8 or 9 groups on the same seven benchmarks, two of which usually contribute two subjects each, and a group is scored on 44 to 76 items in almost every case. The curation guideline states that test subjects are recent systems [1].

Our reference baseline is the Beta-binomial model with a uniform Beta(1,1) prior, which predicts $(k+1)/(m+2)$ for a group with k correct answers among m revealed labels. The official baseline uses a Beta(2,2) prior and is 3.1% worse in the simulator under test weights (Table 1).

Three assumptions link our simulator to this setting.

A1. Public benchmarks with recent subjects, reweighted to the test pool (Section 5), stand in for the private ones. If they differ in other ways, simulated gains may not transfer, in either direction.

A2. Two subjects on one benchmark share most of their items. The feedback cannot confirm this. If it fails, the same-item shift does nothing.

A3. A revealed label is one recorded response, so we simulate it as a Bernoulli draw with the item's success rate.

3 Data

All training data come from measurement-db [3], which is part of the permitted inventory. We kept 166 of its benchmarks with binary responses, 36 times more responses than V3 used, and left out four large or single-subject ones (Appendix B). Subject descriptions are sparse. Only 45% of subject configurations name the model, and 31% give a release date.

4 Method

Text prior. A two-tower network maps the E5 embeddings of the subject text and the item text to a base logit z_{si} for subject s and item i. It is trained with a binomial log loss on all recorded responses, with a free offset per benchmark (Appendix C).

Label sharing. Every revealed label with the target's benchmark alias enters the update, whichever subject it belongs to. On one benchmark, the logit for subject s and item i is

$$\eta_{si} = \gamma z_{si} + c + d_s + u_i, \qquad c \sim \mathcal{N}(0, \tau_c^2), \quad d_s \sim \mathcal{N}(0, \tau_d^2). \tag{2}$$

Here γ scales the text prior, c is a benchmark offset shared by every subject in the draw, d_s is the offset of subject s, and u_i is an item shift. We fit c and all d_s by Newton's method with a Laplace approximation and call this part the *level model*. A label of another subject moves c, and with it every prediction on the benchmark. The item shift u_i moves item i by the residuals of other subjects on the same item, and by the residuals on items whose E5 embeddings are close to it. Each shift is one Newton step for an offset shared by the weighted items (Appendix C). The prediction averages over the posterior variance of $c + d_s$ [10].

Date and name offsets. L4 adds a fixed subject offset fitted on training groups as a linear function of release date, and L5 also adds an ability per model name. Both raise predictions for recent strong systems. The final model does not use them (Section 6.4).

Final submission. The final model, L3, uses a wide level prior ($\tau_c = 2.5$ and $\tau_d = 1$, against 1.2 and 0.6 in the narrow models), both item shifts and the text prior at $\gamma = 0.3$. The archive holds E5, the prior network and the code and runs on CPU. A draw of 8 groups takes 8 to 20 minutes on two virtual CPUs, and no prediction reads a group identifier. Table 1 lists the other models by the parts they add.

5 A simulator calibrated to the hidden test pool

We compare models in a simulator of official draws built from public benchmarks. A draw takes two benchmarks with two subjects each and five with one, as official draws do, and the text prior for each benchmark comes from a network trained without it. We tuned on development draws and report results on 600 fresh draws per setting (Appendix D).

At first the simulator was far off. It gave the official baseline, a Beta-binomial model with a Beta(2,2) prior, an ALC of 0.214, while its 8 official runs averaged 0.187.

The official feedback explains the gap. The baseline predicts one constant q per group, and for a constant the expected calibration error equals $|q - p|$, where p is the group's accuracy. Before any label the baseline predicts $q = 0.5$, so its feedback lists $|p - 0.5|$ for every group. In the baseline

runs available when we chose the model, 58% of test groups had accuracy below 20% or above 80%, against 33% of simulated groups of recent systems.

Test weights reweight simulated groups so that their distribution of $|p - 0.5|$ matches the test pool. With them the simulated baseline scores 0.183, and its curve follows the official one at every budget (Figure 2a). This match is a weak check on its own, because once the distribution matches, the curve follows mostly from group sizes. The side of 50% stays hidden, since flipping every label leaves both scores unchanged. Section 6.4 recovers it.

6 Results

6.1 The text prior transfers almost nothing to unseen benchmarks

In three training variants the best weight of the text prior was the smallest we tried, and ALC improved as the weight shrank (Table 5 in Appendix E). The best variant beat the label-only baseline by 0.1%. The prior mainly acts before the first label, and even there it barely helps. At the chosen weight it lowers the Brier score before any label from 0.250 to 0.249, and at full weight it raises it to 0.258 (Figure 1a). On the platform, the final model's own score before any label is 0.249, again no better than a constant. A second encoder, Qwen3-Embedding-0.6B [7], gave a similarly small gain on a subset, from one set of draws without a standard error, and difficulty ratings from Qwen3-8B [8] did no better (Appendices E and F).

6.2 On the training distribution, shared labels carry the gain

On the training distribution the narrow model lowers ALC by 1.8% against the baseline, and 95% of this remains without the text prior (Figure 1b). Each kind of sharing helps at a different budget. The benchmark offset helps most at 1 to 3 labels, when other subjects on the benchmark add labels that the target lacks. The same-item shift grows with the budget, and the similar-item shift doubles the gain at 31 labels.

6.3 On the test pool, a wide prior wins

Test weights reorder the models (Table 1). On recent subjects without test weights the narrow model L2c is best. With test weights the wide models lead, and L3 gains 3.5% against 2.7% for L2c. L4 and L5 gain more only while groups keep the training side of 50%. With 80% of groups below 50%, the smallest share allowed by the official gap between L4 and L3, L3 keeps 3.1% and L5 falls to 1.4% (third column, Section 6.4). Extreme groups need large offsets, and a narrow prior keeps their predictions near the starting value while labels are few.

Two checks show that L3 gains more than a prior that suits extreme groups. The best Beta prior we tried for this pool, Beta(0.5,0.5), gains only 0.3%. Restricted to the labels of its own group, L3 keeps about half of its gain (1.7% against 3.5%).

The text prior adds 0.6% at the training side of 50% and nothing at 80% below, where the model without it scores -3.2%. Under test weights the real V2 and V3 code trails L3 by 2.5% on simulated draws, while without them it matches L3 (Appendix E).

6.4 Official runs point to most test groups lying below 50%, so the final model drops the offsets

Raising a group's constant prediction from 0.5 to $0.5 + s$ changes its Brier score by $s\,(s - 2(p - 0.5))$, where p is the group's accuracy. The score rises unless $p > 0.5 + s/2$. The date and name offsets raise predictions for recent systems, and the curation guideline says test subjects are recent.

On the platform the offsets raised the Brier score before any label from 0.249 for L3 to 0.275 for L4 and 0.284 for L5. Before any label, every run of L4 and L5 scored worse than every run of L3 (Table 6 in Appendix G). In training data the offsets help, because 62% of groups of systems released from 2025 lie above 50%.

To size the effect we vary the share of simulated groups below 50% (Figure 2b). Even the one-sided 95% lower bound of the official gap between L4 and L3 needs 80% of groups below 50%. L5 beats

Table 1: **Test weights favor wide priors, and once most groups lie below 50%, the date and name offsets hurt.** Change in ALC against the Beta(1,1) baseline in percent, lower is better. Simulator, subjects released from 2025, 600 fresh draws. The first row gives the baseline ALC. The first two columns keep the training share of groups below 50% (38%), and the third sets it to 80%, the smallest share allowed by the official gap between L4 and L3 (Section 6.4). Standard errors are at most 0.2 points. Appendix D adds all subjects.

		Test weights	
Model	Training weights	training side	80% below 50%
Beta(1,1) baseline, ALC	0.210	0.179	0.177
Beta(0.5,0.5)	+1.8	−0.3	−0.5
Beta(2,2), the official baseline	+0.3	+3.1	+3.3
L1c, narrow level model and same item	−1.5	−0.2	+0.0
L2c, narrow, all sharing and text prior	−2.9	−2.7	−1.5
Wide level model	−0.4	−1.8	−1.8
+ same item	−0.8	−2.1	−2.1
+ similar items	−1.5	−2.9	−3.2
+ text prior, $\gamma = 0.3$ (L3, final)	−1.9	−3.5	−3.1
+ date offset (L4)	−2.0	−3.8	−2.0
+ name ability (L5)	−2.1	−4.1	−1.4
L3 with its own group's labels only	−0.1	−1.7	−1.4

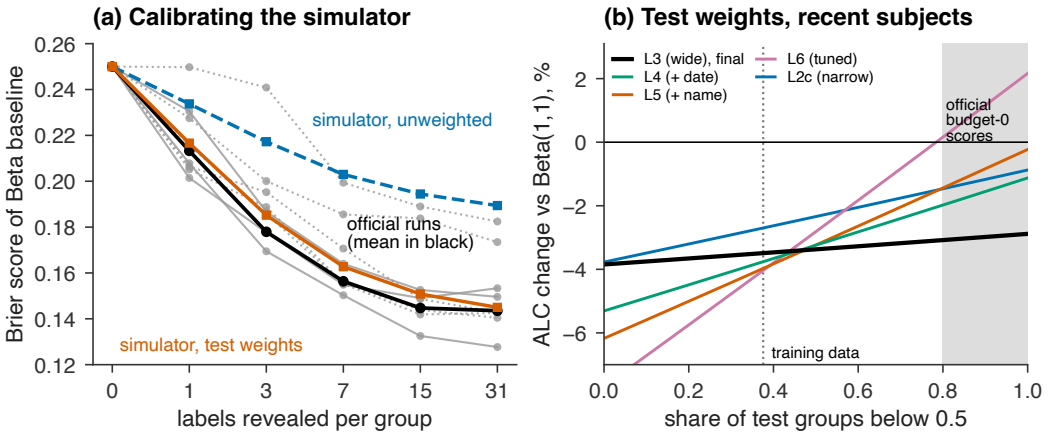


Figure 2: **Test weights reproduce the official baseline curve, and the side of 50% decides whether date and name offsets help.** (a) Brier score per budget of the Beta(2,2) baseline in official runs and in the simulator. (b) ALC change against Beta(1,1) under test weights, recent subjects, as the share of groups below 50% accuracy varies. L6 has larger offsets tuned on development draws (Appendix D). Dotted line, training share. Shaded, shares allowed by the official gap between L4 and L3 (one-sided 95% bound).

L3 only when fewer than 47% lie below. At 80% and above, L5 loses most of its gain while L3 keeps about 3%. We therefore submit L3. No single share fits all three archives exactly, and Appendix D shows the mismatch. Official mean ALC cannot settle the choice. L3 averages 0.190, ahead of L5 but behind the mean of all 8 baseline runs (0.187) and V2 (0.183), and all these means lie within the spread of one run (Table 2).

6.5 Single formative runs cannot rank these methods

The 8 official runs of the baseline have a standard deviation of 0.016 (Appendix G), and two other teams report similar spreads [4, 5]. Each run gets its own draw. Detecting a difference of 0.0039 ALC, the size of the label-sharing gain, takes about 270 runs per archive (two-sided 5% test, 80% power), against 18 paired draws in the simulator. The leaderboard keeps each team's best run, which

Table 2: **Before any label, every L4 and L5 run is worse than every L3 run, while mean ALC does not separate the models.** Official runs, each on its own draw of 8 or 9 groups. $B(0)$ is the mean Brier score before any label. Weight runs set the test weights. L1c and L2c pool builds for the CPU and GPU images. Appendix G lists every run.

Archive	Runs	ALC, mean (s.e.)	$B(0)$
Beta(2,2), weight runs	3	0.178 (0.005)	0.250
Beta(2,2), later runs	5	0.192 (0.008)	0.250
V2	5	0.183 (0.007)	0.249
V3	5	0.186 (0.008)	0.243
L1c	8	0.188 (0.006)	0.250
L2c	3	0.199 (0.010)	0.253
L3, final	3	0.190 (0.008)	0.249
L4	3	0.198 (0.009)	0.275
L5	7	0.193 (0.006)	0.284

for 20 runs lies about 0.030 below the mean, while the final evaluation scores each team once on a common set.

7 Related work

Item response theory models success through subject ability and item difficulty [11, 12], and tinyBenchmarks estimates a benchmark score from a few items with item parameters fitted on that benchmark [13], which a cold start lacks. Observational scaling laws predict scores from capabilities on known benchmarks [14], not item by item on unseen ones. Text-based difficulty estimation works within a domain [15], and amortized calibration predicts item difficulty from question content [18], the role of our text prior. Assessors predict a system's success on an instance [19], and a generic assessor does so for a new model from about 100 reference instances [20]. PAIEC gives at most 31 labels, all from the new benchmark. The level model is a random-intercept logistic model [16, 17].

8 Limitations

The test-pool calibration rests on little data, the feedback of three baseline runs and of the L3, L4 and L5 runs in Table 2, all on the same seven benchmarks. Model selection used it, so our formative runs are not an independent test. Resampling the bins of the test groups (20,000 replicates) keeps L3 ahead of the baseline in every replicate and ahead of L2c in 85%, but ignores groups that recur across runs. The final evaluation draws from the same hidden data [1], but if it weighs benchmarks differently the gain of L3 may shrink.

Under the less extreme groups of the later baseline runs, L3 loses its lead over L2c (Appendix D). If the final pool resembled the training data, L2c would beat L3 by about 1.0%, and if test groups kept the training side of 50%, L5 would beat L3 by 0.6%. The same-item shift relies on assumption A2 but adds only 0.3%. Fresh draws correct for tuning, not for earlier design choices such as the simulator itself.

9 Conclusion

In this cold-start task, labels of other systems on the same benchmark were worth more than a frozen-encoder text prior trained on many benchmarks. Aggregate feedback from a constant baseline told us how extreme the hidden test groups are, and runs of models with shifted predictions pointed to the side of 50% on which most of them lie. Each finding changed our final model. Because single runs are this noisy, formative submissions are best compared over repeated runs.

LLM usage statement

The author directed the work. An LLM-based coding assistant was used mainly to write, debug and run code, and it also helped draft and edit the text. Every result in the tables, figures and text is

printed by a script from saved outputs (Appendix A), and the numbers were checked against those outputs before submission.

References

[1] Stanford AI Measurement Science. The Predictive AI Evaluation Competition at NeurIPS 2026. Rules, data policy, curation guideline and protocol, 2026. `https://aims.stanford.edu/competition`.

[2] Yaroslav Rogoza. Frozen semantic priors with hierarchical evidence adaptation for predictive AI evaluation. Unpublished technical report for PAIEC, 20 October 2026.

[3] AIMS Foundations. measurement-db. Hugging Face dataset, revision 54074453, 2026. `https://huggingface.co/datasets/aims-foundations/measurement-db`.

[4] veseluhha. paiec-2026. Public GitHub repository, accessed 4 November 2026. `https://github.com/veseluhha/paiec-2026`.

[5] Shashwat Srivastav. s2wish-paiec2026. Public GitHub repository, accessed 4 November 2026. `https://github.com/Shashwat-srivastav/s2wish-paiec2026`.

[6] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual E5 text embeddings. A technical report. arXiv:2402.05672, 2024.

[7] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 Embedding. Advancing text embedding and reranking through foundation models. arXiv:2506.05176, 2025.

[8] An Yang and others (Qwen Team). Qwen3 technical report. arXiv:2505.09388, 2025.

[9] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

[10] David J. C. MacKay. The evidence framework applied to classification networks. *Neural Computation*, 4(5):720-736, 1992.

[11] Frederic M. Lord. *Applications of Item Response Theory to Practical Testing Problems*. Lawrence Erlbaum, 1980.

[12] Pedro Rodriguez, Joe Barrow, Alexander Miserlis Hoyle, John P. Lalor, Robin Jia, and Jordan Boyd-Graber. Evaluation examples are not equally informative. How should that change NLP leaderboards? In *ACL*, 2021.

[13] Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin. tinyBenchmarks. Evaluating LLMs with fewer examples. In *ICML*, 2024.

[14] Yangjun Ruan, Chris J. Maddison, and Tatsunori Hashimoto. Observational scaling laws and the predictability of language model performance. In *NeurIPS*, 2024.

[15] Luca Benedetto, Paolo Cremonesi, Andrew Caines, Paula Buttery, Andrea Cappelli, Andrea Giussani, and Roberto Turrin. A survey on recent approaches to question difficulty estimation from text. *ACM Computing Surveys*, 55(9), 2023.

[16] Bradley Efron and Carl Morris. Data analysis using Stein's estimator and its generalizations. *Journal of the American Statistical Association*, 70(350):311-319, 1975.

[17] Andrew Gelman and Jennifer Hill. *Data Analysis Using Regression and Multilevel/Hierarchical Models*. Cambridge University Press, 2007.

[18] Sang Truong, Yuheng Tu, Percy Liang, Bo Li, and Sanmi Koyejo. Reliable and efficient amortized model-based evaluation. In *ICML*, 2025.

[19] José Hernández-Orallo, Wout Schellaert, and Fernando Martínez-Plumed. Training on the test set. Mapping the system-problem space in AI. In *AAAI*, 2022.

[20] Lorenzo Pacchiardi, Lucy G. Cheke, and José Hernández-Orallo. 100 instances is all you need. Predicting the success of a new LLM on unseen data by testing on a few instances. arXiv:2409.03563, 2024.

[21] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *NeurIPS Datasets and Benchmarks*, 2023.

A Reproduction

Paths are relative to the root of the code repository `https://github.com/123yaroslav/paiec2026`, which also holds the sources and data of this report. All CPU steps, the simulator and the runtime tests ran on the Mac with Python 3.12.12, torch 2.14.1, transformers 5.19.0, numpy 2.5.3, pandas 3.0.6, pyarrow 25.0.1 and huggingface_hub 1.33.0. GPU steps ran on Google Colab L4 runtimes with their preinstalled torch and transformers, driven by the colab command-line tool 0.7.4. Two mismatches remain between training and runtime. The prior's training weights give groups with repeated trials less weight than the score does, and training embeddings were stored in float16 while the runtime uses float32. The corpus omits 4 large or single-subject benchmarks and keeps the most-answered items of large benchmarks (Appendix B). Because in-memory caches last one budget, L3 and L5 encode 405 to 588 texts per budget of a draw. On a two-vCPU Colab CPU runtime with one torch thread this costs 0.29 seconds per text and 4.5 seconds per model load, so a run reaches the 8-hour limit only at about 260 groups, against 8 or 9 in a formative run.

1. Download and build the corpus (local CPU). The download fetches benchmark tables at the pinned revision and skips the four benchmarks listed in Appendix B.

```
python - <<'EOF'
import json
from huggingface_hub import snapshot_download
R, REV = "aims-foundations/measurement-db", "54074453da7b9911be0f48e4c560be547bc84d21"
skip = {"mind2web", "naturalreasoning", "swe_smith", "tensortrust"}
keep = [b for b in json.load(open("colab_jobs/binary_benchmarks.json")) if b not in skip]
snapshot_download(R, repo_type="dataset", revision=REV, local_dir="data/mdb_full",
    allow_patterns=["*/formatted_tables/benchmarks.parquet", "*/formatted_tables/subjects.parquet"]
    + [f"{b}/formatted_tables/{t}.parquet" for b in keep for t in ("responses", "items")])
EOF
RAW=data/mdb_full OUT=data/corpus_full RUNTIME_DIR=v5 EMBED=0 SKIP_FETCH=1 python colab_jobs/build_corpus.
     py
```

2. Embeddings, priors and held-out logits (one L4 GPU). We run jobs on Colab through `colab_jobs/ctl.py` (budget check, upload, watch, download with SHA-256 check, stop) and `colab_jobs/supervisor.py` (status file, stall kill, release of idle machines). On the GPU machine, `/content/run` holds `v5/{spec,encoder,textproj,adapt}.py` and the job scripts, and `/content/work` holds the corpus.

```
python /content/run/embed_texts.py
python /content/run/train_eval.py cv --tag A
python /content/run/train_eval.py cv --tag B --bench-offset 1
python /content/run/train_eval.py cv --tag C --bench-offset 1 --dropout 0.3 --wd 0.01 --epochs 2
cd /content/run4 && RUNTIME_DIR=/content/run python export_z.py --tag B   # needs train_eval3.py in /
     content/run3
```

The last command writes `pairs_z.parquet` with held-out logits from five benchmark folds and `prior_B.npz`, the prior trained on all benchmarks (SHA-256 `486b104f...`). The encoder comparison runs `feat_bakeoff.py` and `train_eval2.py` with `-item-keys-file` on the 600-item subset. The subject offsets come from `python colab_jobs/fit_subject_priors.py`, which writes `date_prior.json`, `v5/name_ability.json` and `name_ability_folds.json` and reproduces our files exactly.

3. Test-pool statistics and simulations (local CPU). The first command computes the statistics of Section 5 and Appendix D. The others reproduce Tables 1 and 4, Figure 1, Figure 2b and Section 6.4, the runtime replays and the encoder cost. The baseline script reuses the ladder's draws, and `make_numbers.py` stops if they differ.

```
cd reports/paiec_v5_report
python -B scripts/test_pool_stats.py
python -B scripts/sim_ladder.py --tag recent_conf --min-date 2025-01-01 --seed0 300000  # also all_conf,
     and dev tags with --seed0 0
python -B scripts/sim_baselines.py --tag recent_conf --min-date 2025-01-01 --seed0 300000  # also all_conf
python -B scripts/sim_baselines.py --tag recent_conf --min-date 2025-01-01 --seed0 300000 --only beta,L3,
     L3_own \
  --out baselines_l3                                                                    # also all_conf
python -B scripts/side_of_half.py                                                     # side of 50%
python -B scripts/sim_confirm.py --draws 600 --seed0 100000                           # Figure 1
python -B scripts/extra_stats.py                                                      # judge, name
     ability, replays
```

```
cd ../.. && EX=bertaqa,cellverse,cruxeval,engdesign,erbench,kernelbench,liveaopsbench,researchcodebench,
     tau2_bench
python -B colab_jobs/replay_runtime.py UNZIPPED_V2_DIR colab_jobs/results/replay_v2.parquet --seeds 0:120 \

  --min-date 2025-01-01 --exclude $EX
python -B reports/paiec_v5_report/scripts/record_encoder_texts.py UNZIPPED_L5_DIR texts.json
python reports/paiec_v5_report/scripts/time_encoder.py texts.json encode_timing.json  # CPU machine, next
     to v5/encoder.py
```

The last two commands give the encoder cost of Section 4. The first writes item texts from measurement-db, so its output is not in the repository. The ladder and confirmation scripts import frozen copies of the simulator (`scripts/sim_official_v2_snapshot.py`, SHA-256 `8da09880...`, and `scripts/sim_official_snapshot.py`). The paired simulator rows came from `sim_official.py -emb data/corpus_full/item_emb.npy -draws 120 -min-date 2025-01-01 -exclude $EX -dump colab_jobs/results/sim_dump_recent_ex.parquet` with the variants Beta, L1c, L3, L4 and L5. The dump stores raw per-group ALC and p, so test weights are applied afterwards.

4. Final archive. The first command rebuilds L3 and the last checks it with the validator of the organizers' starting kit, which is not part of our repository. The second builds L5.

```
P=colab_jobs/results/exportB/work/results
python v5/build_zip.py $P/prior_B.npz $P/prior_B.json 2.5 1.0 0.3 dist/v5/L3_wide_prior.zip \
  '{"use_prior": true, "item_exact": 1.0, "item_kernel": [0.03, 3.0]}'
python v5/build_zip.py $P/prior_B.npz $P/prior_B.json 2.5 1.0 0.3 dist/v5/L5_wide_date_name.zip \
  '{"use_prior": true, "item_exact": 1.0, "item_kernel": [0.03, 3.0], "name_scale": 0.8, "date_prior": '"$
     (cat colab_jobs/results/date_prior.json)"'}'
python starting_kit/tools/check_submission_zip.py dist/v5/L3_wide_prior.zip
```

Our L3 archive (Codabench submission 1033014) has SHA-256

`a47150effc0761d29b206ccb548d3c7da6a063e7a22072a352e60dc5b15f7ee7`.

We built it before the offset code entered `model.py`. A rebuild adds that code and the unused name table, and on 360 synthetic calls with 0, 5 and 30 labels it returns identical predictions. The L5 archive has SHA-256 `2d8b7034...` The script `build_zip.py` takes the five E5 files from the V3 archive or, if it is absent, from the hub at the pinned revision, and checks their SHA-256. A rebuild gives identical file contents.

5. Formative runs and this report. The first three commands resubmit an uploaded archive, collect the scores of finished runs and record the runs still waiting in the queue. The others rebuild the numbers, the figures and this PDF.

```
python scripts/resubmit.py L5_wide_date_name.zip 3 --label L5
python scripts/track.py
python scripts/queue_snapshot.py
cd reports/paiec_v5_report && python -B scripts/make_numbers.py && python -B scripts/make_figures.py
pdflatex main.tex && pdflatex main.tex
```

B Training corpus

All training data come from measurement-db at revision `54074453` [3]. Of its 281 benchmarks, 173 have binary responses. We did not download 4 of them (mind2web, naturalreasoning, swe_smith, tensortrust), which have one to three subjects each and include two with more than 500,000 items, and 3 more had no binary rows. Each of the remaining 166 benchmarks keeps at most 4,000 items, the ones answered by the most subjects, and we store the number of recorded trials n and successes k of every subject-item pair. A subject is the exact tuple of its eight fields, so one base model under another harness is a different subject. Of the 14,453 subject configurations, 45% have a model name, 31% a release date and 0.2% a reasoning effort.

C Model details and hyperparameters

Prior network. Items and subjects are embedded with the frozen `intfloat/multilingual-e5-small` encoder [6] at revision `614241f`, with the recipe of

Table 3: **V5 trains on about 36 times more recorded responses than V3.** Groups are subject-benchmark pairs. Accuracies are benchmark-weighted means over groups with at least 80 items.

	V3 training data	V5 corpus
Benchmarks	5	166
Recorded binary responses	1,142,294	40,775,314
Subject-item pairs and items		8,810,555 and 329,666
Subject configurations (with responses)		14,453 (11,420)
Groups with at least 80 items	82 (all sizes)	8,765 on 150 benchmarks
Mean group accuracy, all subjects		0.510 (55% above 0.5)
Mean group accuracy, released from July 2025		0.614 (68% above 0.5)

V3. An item text is its metadata line followed by its task, and a subject text lists its fields. Two towers, each a two-layer perceptron with 256 Gaussian error linear units, map the subject input (E5 vector and release date) and the item input (E5 vector and text length) to h_s and h_i. The logit is $z_{si} = a(h_s) + b(h_i) + \langle Uh_s, Vh_i \rangle / \sqrt{32}$ with linear heads a, b and rank-32 projections U, V. Training minimizes the binomial log loss of the counts (n, k) of every pair, with weight $1/(nT_g)$ for a pair in group g with T_g trials. Each benchmark gets a free offset during training, and the mean offset is folded into the intercept afterwards.

Item shift. The item shift uses residuals of the labeled pairs under the level model. For labeled pair j with subject s_j and item i_j, let $\hat{p}_j = \sigma(\gamma z_j + o_{s_j} + m_{s_j})$, $r_j = y_j - \hat{p}_j$ and $w_j = \hat{p}_j(1 - \hat{p}_j)$, where m_s is the posterior mean of $c + d_s$, o_s the fixed subject offset of L4 and L5 (zero in L3) and y_j the label. For a kernel K with prior variance σ_K^2,

$$u_i = \sum_K \frac{\sigma_K^2 \sum_j K(i, i_j)\, r_j}{1 + \sigma_K^2 \sum_j K(i, i_j)\, w_j}, \tag{3}$$

which is one Newton step from zero for an offset shared by the target item and the labeled items, weighted by K. The same-item kernel $K(i, i') = \mathbf{1}[i = i']$ has variance σ_e^2. The similar-item kernel $K(i, i') = \exp\big((\cos(e_i, e_{i'}) - 1)/h\big)$ on E5 vectors has bandwidth h and variance σ_k^2. With $\eta_{si}^* = \gamma z_{si} + o_s + m_s + u_i$ and v_s the posterior variance of $c + d_s$, the prediction is $p_{si} = \sigma\big(\eta_{si}^* / \sqrt{1 + \pi v_s / 8}\big)$ [10]. L3 uses $\sigma_e^2 = 1$, $h = 0.03$ and $\sigma_k^2 = 3$. The narrow models use $\tau_c = 1.2$, $\tau_d = 0.6$ and $\sigma_e^2 = 2$.

Date and name offsets. The fixed offset o_s adds two subject priors fitted on training groups with at least 80 items. The date prior is $-0.472 + 0.285\, t$, with t the years from 2023-01-01 to the release date clipped to $[-3, 4]$, and -0.093 for an unknown date, fitted by benchmark-weighted least squares to group logit accuracy. The name ability of a normalized model name is the mean, over its training groups, of the group logit minus the benchmark's mean logit, shrunk with a pseudo-count of 2 and clipped to ± 2.5. It enters with weight 0.8, and the submission holds 683 names. In simulation the name ability comes from the four other benchmark folds only. There it has a positive within-benchmark Spearman correlation with group accuracy on 71% of 62 held-out benchmarks with at least five named groups (median 0.18).

Final archive. L3 holds the E5 weights, the prior network and the code (282.6 megabytes) and declares no `models.txt`. It runs on CPU with one prediction worker, which the platform restarts at every budget, so the encoder runs again for each budget. Caches of embeddings, logits and level fits are keyed by content hashes, so no prediction depends on earlier calls.

Text projection. Each text gets the E5 query prefix, and hidden states are mean-pooled under the attention mask and normalized to unit length. Texts longer than 8,192 characters keep a head, a middle and a tail span. After tokenization, inputs longer than 510 tokens keep four spans of 128, 127, 127 and 128 tokens spread evenly over the text. A JSON chat transcript that ends in a user turn keeps up to 512 characters of system messages and the final user message. The item text is the first 300 characters of the item features followed by this projection. The subject text lists name, provider, release date, harness with version, reasoning effort and the first 300 characters of the extras.

Prior training. Subject input is the E5 vector, the years from 2023 to the release date clipped to ± 5 and a missing-date flag. Item input is the E5 vector and $\log(1 + \text{characters})/10$. Hidden width 256, rank 32, dropout 0.1, AdamW [9] with learning rate 2×10^{-3} and weight decay 10^{-4}, one-cycle schedule with 10% warm-up, batch 8,192, 4 epochs, seed 99 for the final prior and the fold index for fold priors.

Leave-benchmark-out grid. The grid crosses $\tau_c \in \{0.8, 1.2, 1.8, 2.5, 3.5\}$ with $\tau_d \in \{0.5, 0.8, 1.2\}$ and $\gamma \in \{0.3, 0.45, 0.6, 0.75, 0.9\}$, 75 configurations in total. The first run used γ from 0.7 upward and was 1.32% worse than Beta-binomial.

Simulator grids. Newton iterations stop after 25 steps or when the largest step is below 10^{-7}. All three grids of the official-structure simulator used 400 development draws and include the Beta(1,1) baseline. Here $\gamma = 0$ means no text prior. Grid 1 (no test weights, all subjects, 487 configurations) crossed $\tau_c \in \{0.9, 1.2, 1.6\}$, $\tau_d \in \{0.6, 0.8, 1.1\}$, $\sigma_e^2 \in \{2, 4, 8\}$, $h \in \{0.02, 0.03, 0.05\}$, $\sigma_k^2 \in \{1, 1.5, 3\}$ and $\gamma \in \{0, 0.3\}$. Its best difference was -0.00463 (standard error 0.00028), and it chose the narrow configuration. Grid 2 (earlier test-weight shares, all subjects, 289 configurations) crossed $\tau_c \in \{2, 2.5, 3.5, 5\}$, $\tau_d \in \{0.8, 1, 1.4, 2\}$, $\sigma_e^2 \in \{1, 2, 4\}$, $\sigma_k^2 \in \{1.5, 3, 6\}$ at $h = 0.03$, and $\gamma \in \{0, 0.3\}$. It chose the wide level prior of L3. Grid 3 (earlier test-weight shares, subjects from 2025, 326 configurations with L5) crossed $\tau_c \in \{2, 2.5, 3.2\}$, $\tau_d \in \{0.8, 1, 1.3\}$, $\sigma_e^2 \in \{0.5, 1, 2\}$, $\gamma \in \{0.2, 0.3, 0.45\}$, name weight $\{0.8, 1.2\}$ and date weight $\{1, 1.5\}$ at $\sigma_k^2 = 3$ and $h = 0.03$. Its best configuration is L6. Top lines are in `colab_jobs/results/sim_testw_grid2.log` and `sim_grid3_recent.log`. Of the 600 draws of the unweighted confirmation run, 119 hold 9 groups, 385 hold 8 and 96 hold 7.

Judge prompt. The system message reads "You estimate how hard a task is for current frontier AI systems. Read the task and rate its difficulty on a scale from 1 to 9, where 1 means almost every AI system solves it correctly and 9 means almost none does. Answer with a single digit only." The user message is "Task:" followed by the item text, truncated to 1,024 tokens of head and tail, and "Difficulty (1-9):". Thinking mode is off.

D Simulator, test weights and the side of 50%

Baseline. The Beta-binomial baseline predicts $(k + a)/(m + 2a)$ for every evaluation item of a group with m revealed labels and k successes. Simulations use $a = 1$ and the official archive uses $a = 2$, which is worse by 0.00023 ALC in simulation (standard error 0.00019).

Leave-benchmark-out simulation. We split the 166 benchmarks into 5 random folds, train the prior on four and simulate on groups of the fifth with at least 80 items, subsampled to 80 to 160 items. Draws of the level model collect groups up to 1,000 items, 60 per fold, from 8,765 held-out groups.

Official-structure simulator. A draw picks 2 benchmarks whose two sampled subjects share 88 to 152 items and 5 benchmarks with one subject, and drops groups beyond 1,000 items, so most draws hold 8 groups. Each group is split in half independently. Base logits come from the prior trained without the pair's benchmark fold. Options restrict subjects to those released from 2025 (56 benchmarks) and apply test weights. Seeds 0 to 599 serve development, the tuning grids used 400 of them, and 600 seeds from 100,000, 200,000 and 300,000 serve one confirmation run each. Standard errors treat the draw as the unit.

Runtime replays and formative runs. We ran the real V2 and V3 submission code on the simulator's draws (recent subjects, seeds from 0) and paired each group with the simulator's L3, L5 and Beta-binomial rows. These draws exclude the nine benchmarks that V2 or V3 trained or tuned on. On the platform each submission is scored on a new random draw, so we resubmit an archive several times and report the mean and standard error across runs.

Test weights. For a group whose prediction is one constant q, the Brier score is $p(1-p) + (q-p)^2$ and the expected calibration error is $|q - p|$. The official baseline therefore gives $|p - 0.5|$ at budget 0 for every group, and $p(1-p) = B_{31} - \mathrm{ECE}_{31}^2$ agrees with it to 10^{-6}. The shares of $|p - 0.5|$ in the bins $[0, 0.1)$, $[0.1, 0.2)$, $[0.2, 0.3)$, $[0.3, 0.4)$ and $[0.4, 0.5]$ are 0.115, 0.077, 0.231, 0.308, 0.269.

The tuning grids used an earlier estimate (0.115, 0.077, 0.192, 0.308, 0.308) that placed one group in a different bin, and all confirmation runs use the shares above. With test weights the simulated Beta(2,2) baseline scores 0.1826 on all subjects and 0.1841 on recent subjects, against 0.2136 and 0.2105 without them. The three weight runs average 0.1778 (standard error 0.0049). Later runs, made after the model choice, scored 0.2190, 0.1777, 0.1835, 0.2017, 0.1790, and all 8 runs average 0.1868 (standard error 0.0057). Under the shares of the 44 less extreme evaluations of the later runs, the gaps of L3 to Beta(1,1) and to L2c are -0.0049 and $+0.0004$. Official runs of V2 (mean 0.1825, 5 runs) and V3 (0.1862, 5) lie within one standard error of the Beta(2,2) mean. Only such aggregate statistics and per-archive scores enter model selection.

Absolute results. Table 4 gives the first two columns of Table 1 as absolute ALC differences and adds all subjects and some variants. In the wide level model, sharing only the benchmark offset is worth 0.0022 (standard error 0.0001) under test weights on recent subjects and needs no shared items, and the same-item step adds 0.0006.

Table 4: **Simulator results in absolute ALC.** Official-structure simulator, the same 600 confirmation draws (seeds from 300,000) in each population. The first row is the Beta(1,1) ALC, other rows are differences from it, lower is better. Standard errors are 0.0001 to 0.0003 per cell.

Subjects	all	all	recent	recent
Test weights	no	yes	no	yes
Beta-binomial ALC	0.2136	0.1769	0.2099	0.1786
Beta-binomial, Beta(0.5,0.5)	$+0.0043$	-0.0006	$+0.0037$	-0.0006
Beta-binomial, Beta(2,2), official prior	$+0.0000$	$+0.0057$	$+0.0006$	$+0.0055$
L1c, narrow level + same item	-0.0029	$+0.0005$	-0.0033	-0.0004
L2c, narrow, all sharing + E5 prior	-0.0043	-0.0027	-0.0062	-0.0049
Wide level model	-0.0002	-0.0027	-0.0008	-0.0032
+ same item	-0.0009	-0.0032	-0.0016	-0.0038
+ similar items	-0.0021	-0.0045	-0.0032	-0.0052
+ E5 prior, $\gamma = 0.3$ (L3, final)	-0.0023	-0.0049	-0.0039	-0.0063
+ date prior (L4)	-0.0023	-0.0052	-0.0043	-0.0068
+ name ability (L5)	-0.0021	-0.0051	-0.0044	-0.0072
L5 without the E5 prior	-0.0021	-0.0049	-0.0039	-0.0065
Wide level model, own labels only	$+0.0017$	-0.0009	$+0.0013$	-0.0010
L3, own labels only	$+0.0009$	-0.0023	-0.0003	-0.0031
L6, best of a grid on recent dev draws	-0.0023	-0.0045	-0.0051	-0.0075

Side of 50%. On the platform the budget-0 Brier score was 0.2489 for L3 (standard error 0.0013, 27 group evaluations), 0.2746 for L4 (0.0065) and 0.2836 for L5 (0.0066). We reweight simulated groups within each $|p - 0.5|$ bin so that a share π lies below 0.5. At the training share of 0.38 the simulated budget-0 gap between L4 and L3 is -0.003, and even at $\pi = 1$ it reaches only 0.023, below the official 0.026 (standard error 0.007). The one-sided lower 95% bound of the official gap needs $\pi \geq 0.80$ (two-sided, 0.74). No single share fits all three archives. The official L4 and L5 scores alone both point to π near 0.87, and the L4 score alone needs $\pi \geq 0.70$, but the simulated L3 rises to 0.2554 at $\pi = 0.80$ against its official 0.2489, which alone points to 0.54. The gap between L4 and L3 removes the part they share, the text prior, so we use it. L5 beats L3 only for $\pi < 0.47$. L3 is better by 0.0029 at $\pi = 0.80$ and by 0.0047 at $\pi = 1$ (standard errors about 0.0001), and the tuned L6, with larger offsets, falls behind Beta-binomial for $\pi > 0.78$. The difference of L3 from Beta(1,1) stays between -0.0069 and -0.0051 over the whole range. On all subjects L5 and L3 differ by at most 0.0003, because older subjects get small offsets. Figure 2b has standard errors of at most 0.0005.

E Leave-benchmark-out results and runtime replays

With test weights, L3 beat the real V2 code by 0.0045 (standard error 0.0013, 120 draws, 974 groups) and V3 by 0.0045 (0.0009, 120 draws). Without weights the differences were -0.0003 (0.0009) and $+0.0004$ (0.0008). V2 was close to Beta-binomial under test weights (-0.0013, standard error 0.0013) and better without them (-0.0029). So V2 and V3 match L3 on the training distribution but fall behind it on the extreme test pool, where their narrow priors lose the gain over Beta-binomial. A

Table 5: **In every variant the best prior weight is the smallest one tried.** Leave-benchmark-out ALC with the level model, best (τ_c, τ_d) per row, lower is better. Each block has its own draws and baseline. Variant C adds dropout 0.3, weight decay 0.01 and 2 epochs to B.

Data	Prior	ALC	vs Beta-binomial
Full corpus	Beta-binomial, Beta(1,1)	0.21244	
	B, $\gamma = 0.3$	0.21217	-0.13%
	B, γ from 0.45 to 0.9	0.21258 to 0.21503	
	A (no benchmark offsets), $\gamma = 0.3$	0.21260	$+0.08\%$
	C (more regularization), $\gamma = 0.3$	0.21229	-0.07%
600-item subset	Beta-binomial, Beta(1,1)	0.21801	
	E5 prior, $\gamma = 0.3$	0.21684	-0.54%
	Qwen3-Embedding-0.6B prior, $\gamma = 0.3$	0.21651	-0.68%

replay of the L5 runtime on 3 draws (23 groups) matched the simulator within 0.0044 ALC per group on average, and a rebuild of L3 with that code returns identical predictions (Appendix A).

F The LLM difficulty judge

Following the use of LLMs as graders [21], Qwen3-8B [8] rated item difficulty from 1 to 9 as the expected digit of its next-token distribution, with the prompt of Appendix C. It rated 13,880 of 18,952 sampled items before we stopped the run to save compute, so long items are missing. The within-benchmark Spearman correlation between rating and item accuracy had median -0.062 and was negative, the useful direction, on 63% of 131 benchmarks. Between benchmarks it was -0.049. In the simulator on rated items with test weights, the best weight of 0.2 on the standardized rating improved ALC by only 0.00035 and hurt budget 0.

G Formative runs

Table 6 lists every finished formative run of our team. V1 and the V3 variant with an uncertainty band are earlier entries not described here.

H Training sources, pretrained models and compute

Training sources. All training and development data come from measurement-db at revision `54074453da7b9911be0f48e4c560be547bc84d21`, restricted to the 166 binary benchmarks of Section 3. No other data source was used for training, tuning or simulation. The test weights use aggregate statistics of our own formative feedback (Section 5). Earlier entries V2 and V3 used four and five benchmarks of the same inventory [2].

Pretrained models. No model was fine-tuned, and no paid API was used. Table 7 lists every pretrained model we ran.

Compute. GPU work ran on Google Colab L4 instances, and one Colab CPU runtime timed the encoder. Together they used 25.9 compute units of our own. Corpus building, simulation, replays, packaging and this report ran on an Apple M4 laptop.

Table 6: **All finished formative runs.** Means over groups of one draw each. B_0 and B_{31} are mean Brier scores at budgets 0 and 31.

Submission	Archive	Groups	Benchmarks	ALC	B_0	B_{31}
1003760	Beta(2,2) baseline	9	7	0.1867	0.2500	0.1496
1003791	V1 hierarchical prior	8	7	0.1911	0.2511	0.1612
1004759	V2	9	7	0.1695	0.2536	0.1330
1004796	V3	9	7	0.1711	0.2401	0.1478
1004970	V3 with uncertainty band	9	7	0.1876	0.2341	0.1644
1032873	V2	8	7	0.1790	0.2480	0.1484
1032874	V2	9	7	0.2013	0.2472	0.1699
1032875	V2	8	7	0.1644	0.2452	0.1303
1032876	V2	9	7	0.1983	0.2502	0.1802
1032877	V3	9	7	0.2172	0.2709	0.1843
1032878	V3	9	7	0.1861	0.2421	0.1594
1032879	V3	9	7	0.1857	0.2300	0.1565
1032880	V3	9	7	0.1709	0.2297	0.1396
1032881	Beta(2,2) baseline	9	7	0.1698	0.2500	0.1277
1032882	Beta(2,2) baseline	8	7	0.1769	0.2500	0.1533
1032902	L1c, GPU image	9	7	0.1803	0.2500	0.1528
1032903	L1c, GPU image	8	7	0.2102	0.2500	0.1815
1032904	L1c, GPU image	8	7	0.1919	0.2500	0.1627
1032928	L2c, GPU image	9	7	0.1795	0.2529	0.1496
1032929	L2c, GPU image	8	7	0.2128	0.2548	0.2006
1032930	L2c, GPU image	8	7	0.2034	0.2527	0.1736
1032960	L1c	9	7	0.1578	0.2500	0.1123
1032962	L1c	8	7	0.2052	0.2500	0.1819
1032963	L1c	9	7	0.1840	0.2500	0.1465
1033014	L3, final	9	7	0.2053	0.2511	0.1856
1033015	L3, final	9	7	0.1860	0.2491	0.1661
1033016	L3, final	9	7	0.1775	0.2466	0.1451
1033024	L4	8	7	0.2066	0.2723	0.1703
1033025	L4	9	7	0.2068	0.2768	0.1735
1033026	L4	9	7	0.1807	0.2744	0.1528
1033031	L5	9	7	0.1774	0.2870	0.1473
1033033	L5	9	7	0.1915	0.2878	0.1550
1033034	L5	9	7	0.2165	0.2790	0.1771
1034238	L5	8	7	0.1909	0.2911	0.1261
1034630	Beta(2,2) baseline	9	7	0.2190	0.2500	0.1824
1034631	Beta(2,2) baseline	9	7	0.1777	0.2500	0.1404
1034632	L5	8	7	0.1802	0.2793	0.1525
1034633	L5	9	7	0.2085	0.2791	0.1901
1034634	L5	8	7	0.1837	0.2821	0.1613
1034661	Beta(2,2) baseline	9	7	0.1835	0.2500	0.1429
1034668	L1c	9	7	0.1985	0.2500	0.1683
1034689	L1c	9	7	0.1789	0.2500	0.1352
1034696	Beta(2,2) baseline	9	7	0.2017	0.2500	0.1735
1034704	Beta(2,2) baseline	8	7	0.1790	0.2500	0.1426

Table 7: **Pretrained models used in this work.** Only E5 runs inside the submission.

Model	Revision	Use
`intfloat/multilingual-e5-small`	`614241f622f53c4eeff9890bdc4f31cfecc418b3`	frozen encoder in training and in the submission
`Qwen/Qwen3-Embedding-0.6B`	`97b0c614be4d77ee51c0cef4e5f07c00f9eb65b3`	encoder comparison only
`Qwen/Qwen3-4B`	`1cfa9a7208912126459214e8b04321603b3df60c`	feature extraction, stopped before it finished
`Qwen/Qwen3-8B`	`b968826d9c46dd6066d109eabc6255188de91218`	difficulty judge